

Imputando valores faltantes

Olá! No vídeo anterior a esta aula, trabalhamos com um conjunto de dados que tem alguns dados faltantes.

Alguns dados não foram escritos de maneira satisfatória para um conjunto de dados e essa ausência de alguns dados pode causar problemas em análises e em algoritmos de aprendizado de máquina.

Neste vídeo, iremos tratar esses conjuntos de dados de modo que esses dados faltantes passem a não existir mais.

Dessa maneira, os algoritmos não terão nenhum tipo de problema com ausência de dados.

Para tanto, primeiramente, iremos carregar novamente o nosso conjunto de dados de modo a identificar os valores faltantes.

Portanto, iremos utilizar novamente uma linha, uma célula utilizando a variável "data", ou dados, "`= pd.read_table`" passando a string como parâmetro da função "`read_table`": "`('fruit_data_with_colors_miss.txt')`".

Ou seja, até então, temos uma linha igual à da leitura dos dados efetuada anteriormente:

```
"data= pd.read_table('fruit_data_with_colors_miss.txt')".
```

Passe o nome do arquivo como string, entre aspas simples (') ou duplas (") no caso, estou utilizando aspas simples.

Então, a diferença que iremos ter é que, após essa string, iremos passar um segundo parâmetro.

Então, uma vírgula (,) ao final da string após a aspa que fecha o nome do arquivo iremos utilizar

"na_values", que é um parâmetro da função "`read_table`" para indicar quais os dados faltantes

do conjunto de dados.

Então: "`na_values=[]`" iremos passar uma lista de valores, que são os valores faltantes e, de acordo com o conjunto de dados, temos dois valores faltantes.

Então, nesta lista, iremos passar como primeiro parâmetro a string contendo apenas um ponto, e o segundo parâmetro dessa lista, seu segundo elemento, será uma string contendo um ponto de interrogação, que seria o outro valor faltante.

Então, a nossa linha completa ficou: `data = pd.read_table(" o primeiro parâmetro do "read_table" seria a string: "fruit_data_with_colors_miss.txt"`

Para o segundo parâmetro, ou seja, separando com uma vírgula, teremos: `"na_values="` a uma lista cujo primeiro elemento é uma string ponto, e o segundo elemento desta lista é a string ponto de interrogação. Fecha o parêntese final `)"` para indicar que a `"read_table"` foi finalizada com os dois parâmetros o nome do arquivo, o conjunto de dados e a lista de valores, que indica valores ausentes em nosso conjunto de dados.

Ao executar esta célula notaremos que a variável `"data"` terá o mesmo conjunto de dados, porém com a diferença de ela saber quais são os valores faltantes que seria o ponto e a interrogação neste conjunto de dados.

Cada conjunto de dados pode indicar os valores faltantes diferentemente.

Na linha abaixo, na célula abaixo iremos imprimir o nosso conjunto de dados:

`"data"`, e executando esta linha apenas com a variável `"data"` do nosso conjunto de dados iremos imprimir nosso conjunto de dados novamente.

E, desta vez, onde havia valores faltantes representados por ponto e interrogação teremos os caracteres `"NaN"`, que significam, em inglês, `"not a number"`, ou seja, `"não é um número"`.

Isso indica um valor faltante no nosso conjunto de dados.

Este conjunto de dados agora possui uma informação para o Python que indica onde temos os dados faltantes, ou seja, onde há dados que não existem porque o ponto final e ponto de exclamação poderiam ser tratados como dados de verdade.

O computador, o Python, não tem como saber que o ponto de exclamação e o ponto de interrogação são dados faltantes.

Cabe a nós, ao programador, indicar quais são esses dados faltantes como eles estão representados no conjunto de dados. Uma vez que refizemos a leitura do conjunto de dados utilizando os `"na_values"` para passar a lista de valores faltantes teremos `"NaN"` onde existiam

esses valores faltantes.

Na próxima célula, veremos como imputar dados a esses valores faltantes. Esse termo, `"imputar dados"`, se referem a colocar a injetar dados nesses locais.

Então, iremos efetuar o seguinte comando na próxima célula: `data.fillna(0)`.

Ou seja, `"data"` é o nosso conjunto de dados; ponto significa que iremos utilizar uma função neste conjunto de dados, e `"fillna"` que, traduzindo, é algo parecido com

"complete os não-disponíveis", e "(0)" significa que iremos imputar o valor "0" aos valores faltantes, e iremos executar esta linha.

Uma vez executada esta linha, não existem mais valores faltantes no nosso conjunto de dados

uma vez que os valores que existiam até então foram substituídos pelo "0".

Porém, substituir por um valor e manter a coesão do conjunto de dados é bem difícil uma vez que não temos nenhuma garantia de que este valor imputado será equivalente ou razoavelmente parecido com a distribuição dos dados.

Tomando apenas a coluna das massas, temos as primeiras linhas, como massa "192.0", uma fruta de massa 192; na segunda linha, massa "180.0"; na terceira, massa "176.0"; na quarta, massa "0.0". Isso é estranho, foge ao padrão normal.

Mesmo entre as tangerinas, que vai a partir da quarta linha até a oitava linha, as massas variam

entre "84.0", "80.0", outra de "80.0" e uma de "76.0".

Ou seja, esse valor "0.0" está bem distante do que parece ser o padrão das frutas.

Então, deve-se tomar cuidado ao utilizar um valor constante para escrever os valores faltantes.

Uma técnica um pouco diferente que podemos utilizar é aplicar a média dos dados selecionados.

Podemos verificar a média utilizando a função "describe".

Na próxima célula, podemos utilizar "data.describe()".

Ao executar esta linha, teremos a análise estatística básica do nosso conjunto de dados. Temos a informação que a média da massa dada na segunda linha e coluna desse nosso conjunto de dados com o "describe" temos que sua média é: "164.448276".

Então, como utilizar essa média da massa nesta coluna? Poderíamos utilizar a função "fillna"

e aplicar o valor "164.0", porém este valor seria replicado nas outras colunas e gostaríamos que cada coluna tivesse sua média calculada separadamente.

Então, podemos utilizar uma abordagem um pouco diferente para tentar resolver este problema.

Iremos efetuar o seguinte código: "data.fillna(data.mean())", mean de média.

Ao executar este comando, nesta linha, temos que a média de cada uma das colunas foi utilizada para imputar os valores faltantes.

Ou seja, na massa, onde estava ausente o valor na quarta linha foi colocada a média de "164.44" assim como pudemos verificar com a função "describe".

Na linha 8, ou a nona linha, temos a altura, que era um dado faltante, virou "7.691379", que é exatamente a média de altura. Assim, utilizando a média do próprio conjunto de dados, o Python e o Pandas irão identificar automaticamente que a média é independente para cada coluna.

Uma única situação que ainda temos, é que na coluna "fruit_subtype", ou subtipo de fruta o valor faltante não foi alterado. Uma vez que esta coluna é formada por strings é inviável utilizar

a média desta coluna para imputar um valor faltante.

Outra possibilidade pode ser a de utilizar o valor que mais se repete nesta coluna.

Em outra linha, em outra célula do nosso Jupyter Notebook, iremos utilizar o seguinte comando:

```
"data ['fruit_subtype']", ou seja, estamos indicando que iremos utilizar a coluna "fruit_subtype" dos dados "data": "data ['fruit_subtype'].value_counts().argmax().
```

Dessa maneira, faremos a contagem dos elementos da nossa coluna e iremos ter o elemento que foi indicado mais vezes no conjunto de dados.

Executando esta linha, teremos o resultado "turkey_navel" que é o subtipo que mais se repetiu.

É um subtipo mais simples de aparecer e de ser colocado ou, no caso, por estarmos utilizando strings não é problema, também, preencheremos um valor faltante utilizando alguma outra string,

como "missing" que seria uma string para identificar se tem algo faltando.

Esta também é uma possibilidade. Para inserir no nosso conjunto de dados porque, até então, só apresentamos como o conjunto de dados ficaria se os valores fossem imputados das maneiras que vimos até então. Então, vamos efetivamente armazenar, do nosso conjunto de dados, esta imputação dos dados.

Na próxima célula, iremos utilizar nossa variável do conjunto de dados "data = data.fillna(data.mean())", iremos armazenar nossa variável "data".

Ao executar esta linha, teremos que nosso conjunto de dados, o "data" agora terá, pelo menos em suas colunas de números a informação com a média das colunas faltantes onde havia valores faltantes.

Para aplicar o valor faltante na coluna "fruit_subtype" iremos utilizar o seguinte comando:

`data[fruit_subtype]=data[fruit_subtype].fillna(data[fruit_subtype].value_counts().argmax())`), utilizando a técnica para reaver o elemento mais repetido na coluna, ou seja, ao executar esta linha, iremos ter o nosso conjunto de dados com os dados faltantes desta coluna em específico tendo o valor que mais se repete.

A nossa linha ficou bem grande onde estamos utilizando: `"data[fruit_subtype]"`, assim, trabalhamos com só uma coluna do nosso conjunto de dados `"data[fruit_subtype]"` novamente `".fillna"`, para completar nossos dados faltantes e imputá-los `"(data[fruit_subtype]"` novamente no nosso índice `".value_counts().argmax())"` e fecha o parêntese, `)"`, do `"fillna"`.

Dessa forma, nosso conjunto de dados agora terá, na coluna `"fruit_subtype"` em seus dados faltantes, o valor que mais se repete nesta coluna.

Trabalhamos com imputação de dados sejam eles strings ou valores numéricos que é uma técnica eficiente.

Não é a melhor técnica que podemos utilizar, mas é uma técnica que resolve a maioria dos casos.

Então, como abordagem introdutória sobre a imputação de dados faltantes utilizar a média é uma boa solução ou utilizar, no caso, o valor que mais se repete de uma coluna de strings também é uma possibilidade.

Com isso, encerramos este vídeo sobre imputação de dados.

Nos encontramos no próximo vídeo.

Até lá!